

A First Look at the Mobile Driving License (mDL) Standard and its Real-world Usage

Zeyu Lei	Güliz Seray Tuncay	Abdullah Imran	Z. Berkay Celik	Antonio Bianchi
Purdue University	Google	Purdue University	Purdue University	Purdue University
USA	USA	USA	USA	USA
zeylei97@gmail.com	gulizseray@google.com	imran8@purdue.edu	zcelik@purdue.edu	antonioib@purdue.edu

Abstract

The mobile driving licenses (mDL) standard enables the replacement of physical licenses with secure digital copies. This standard has been progressively adopted by more and more entities, including some U.S. states, and implemented by mobile app developers.

As this standard deals with sensitive information, its correctness is of paramount importance. Unfortunately, no previous work has performed a systematic analysis to ensure the correctness of the standard and its implementations.

In this paper, we present the first systematic security analysis of the mDL standard. Our analysis starts with extracting the critical implementation requirements from the specification. We then model multiple mDL usage scenarios by building a dedicated verification framework based on ProVerif. Our formal model highlights potential attacks that could occur if requirements are incorrectly implemented and generates step-by-step attack traces for these attacks. Starting from these attack traces, we then built mDLVerif, a black-box differential dynamic analysis tool, to analyze mDL reader apps in the market and evaluate their compliance with the mDL standard. Our study proves the overall correctness of the specification. However, we find that fulfilling the identified mDL implementation requirements is challenging for developers. Our formal model also highlights that these mistakes can lead to severe security implications, such as authentication failure and unauthorized access to restricted resources. In fact, our automated app analysis reveals that all six apps in our dataset implement the mDL standard in a way that is not compliant with the specification.

CCS Concepts

• **Security and privacy** → **Authentication**; **Formal security models**; *Mobile and wireless security*.

Keywords

Mobile Driving License, Mobile Security, Formal Verification, Dynamic Analysis, Implementation Security, Authentication

ACM Reference Format:

Zeyu Lei, Güliz Seray Tuncay, Abdullah Imran, Z. Berkay Celik, and Antonio Bianchi. 2026. A First Look at the Mobile Driving License (mDL) Standard and its Real-world Usage. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '26)*, June 1–5, 2026, Bangalore, India. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3779208.3785416>



This work is licensed under a Creative Commons Attribution 4.0 International License. *ASIA CCS '26, Bangalore, India*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2356-8/26/06

<https://doi.org/10.1145/3779208.3785416>

1 Introduction

In the U.S., over 90% of the population possesses a driving license, and over 40 states have implemented barcode-readable driver's licenses [21]. Such physical documents, however, pose inherent security risks due to their susceptibility to loss or counterfeiting. While the document holder (i.e., an individual to whom a document is issued) hands over the physical documents to the verifier (i.e., a person or organization who verifies the document), not only may the verifier seize or damage the credentials, but also all information on the physical documents would be fully accessible to the verifier [14]. For instance, during alcohol purchases, presenting physical documents would unnecessarily disclose personal addresses.

To mitigate these concerns, the Mobile Driving License (mDL) standard was recently introduced by the International Organization for Standardization (ISO) under ISO/IEC 18013-5:2021 [23]. An mDL is a digital representation of a physical driver's license or identification card securely stored and presented on a mobile device through an app without physical possession of the credential to the verifier. This standard replaces the need for traditional physical documents, and, according to the Secure Technology Alliance (STA), at least 30 states in the U.S. are actively pursuing mDL initiatives.

To the best of our knowledge, no prior work has conducted a systematic security analysis of the mDL standard, its specifications, real-world use cases, and its implementations used by mobile apps. While a line of prior work has performed formal analyses of protocols similar to mDL, specifically the U-Prove protocol [33, 34], these studies are not directly applicable to the mDL standard. This is due to the mDL standard utilizing cryptographic techniques and communication patterns specific to its design, which were not considered in earlier research.

To bridge this critical gap, we first analyzed the mDL specifications to identify critical implementation requirements (i.e., security checks). Our systematic approach involved a dual strategy: we manually analyzed the mDL specifications while also using LLMs to automatically extract critical implementation requirements. Both methods independently converged on the same set of seven implementation requirements for mDL reader apps, providing cross-validation of our findings.

Building upon these identified requirements, we constructed a formal model of the mDL protocol (M_{mDL}) supporting the verification of the security properties specified in the mDL standard. To obtain a complete and sound unified model, we analyzed and integrated documentation from three distinct documents: the mDL standard [23], the mDL white paper [1], and the mDL usage guidelines [2]. Then, to correctly express the potential real-world usage of mDL, we integrated the verification of 11 distinct use cases of this protocol, as outlined in the mDL white paper [1], into the M_{mDL} .

During the modeling process, we developed a modular model construction framework based on ProVerif [11] that efficiently verifies multiple scenarios. Our framework is designed to be *flexible* and *scalable*, allowing us to express various use cases and security requirements of the mDL standard. It is also *extensible* to accommodate future use cases as the standard evolves. Using the framework, we verified M_{mDL} and showed that its basic security properties hold, and we demonstrated that all the security goals are achieved in each of the 11 real-world usage scenarios.

Then, to understand how attacks can be conducted when implementation requirements are not correctly implemented in mDL reader apps, we modified the original model (M_{mDL}) by intentionally disabling one security check at a time. This resulted in a revised model M'_{mDL} , which generates step-by-step attack traces that demonstrate the exploitation path when a security check is missing. These traces reveal a series of attacks that can be conducted when a security check is missing, even in cases where the attack method is not immediately apparent (see Section D).

Leveraging the insights from our formal modeling, we conducted a comprehensive security analysis of real-world mDL reader apps to evaluate their implementation. To ensure our analysis was systematic, thorough, and produced fully reproducible results, we developed mDLVerif [6], a dynamic testing framework for black-box security analysis of mDL reader apps.

To determine whether a reader app accepts or rejects a malicious mDL document, mDLVerif conducts differential dynamic analysis. First, it sends a compliant mDL document to the reader app, recording its visual state as the baseline. Then, starting from the attack traces obtained from M'_{mDL} , mDLVerif crafts malicious mDL documents and sends them to the tested reader app. Finally, it compares the visual state of the reader app after processing a malicious mDL against the baseline, to determine whether the tested reader app can detect the document's noncompliance.

This approach relies on the observation that mDL reader apps should present a different UI based on the verification results, so that an app user can distinguish between compliant and non-compliant mDLs. As such, during comparison, if the reader app displays different UI elements in response to malicious inputs; it likely indicates a correct implementation of the security checks. In fact, in this case, a human verifier could rely on visual cues to differentiate between compliant and non-compliant mDL documents. Conversely, if the app responds identically to both compliant and non-compliant mDL documents, human verifiers who use these reader apps will receive no visual indication to distinguish between legitimate and potentially fraudulent mDL documents, strongly suggesting the presence of a vulnerability.

Using this approach, we analyzed six mDL reader apps and found that all six apps had implementation issues leading to the violation of at least one specification requirement. For example, GET Mobile Verify [19] failed to ensure the correctness of one of the properties of the mDL standard, which is the document type itself. This potentially allows an attacker to present a different type of legitimate document (e.g., a vaccination card), forge its document type to appear as a driver's license, and still have it accepted by the reader app. In another case, we found that Identity Credential [20], which is Google's reference mDL implementation, verifies the data items provided by the mDL holder even when they do not

provide their portrait. This implementation choice directly contradicts what is mandated by the mDL standard for reader apps, and it could allow an attacker to present someone else's document. In fact, since no portrait is shown, the verifier would have no way to visually confirm whether the person presenting the document is its rightful owner. The same issue was also found in other mDL reader apps, such as Mobile ID Verify, an mDL reader app developed by Idemia R&D. We reported our findings to the developers of these six apps. Google, GET Group, and CredenceID acknowledged our findings, with Google and GET Group implementing fixes based on our suggestions.

In summary, this paper makes the following contributions:

- **Systematic Study of the mDL Standard.** We are the first to systematically analyze the security of the ISO/IEC 18013-5:2021 (mDL) standard, employing formal methods and program analysis to assess both its specifications and implementations.
- **Identification of Critical Implementation Requirements.** We identify and validate seven critical implementation requirements for mDL reader apps through a dual approach of manual specification analysis and automated LLM-based extraction, bridging the gap between specification and secure implementation.
- **Formal Verification Framework.** We create a formal verification framework [6] for mDL that automatically generates models to handle different queries, variations, and scenarios. Using this framework, we verify the standard's basic security properties and analyze 11 real-world usage scenarios, along with the risks of incorrect implementation.
- **Dynamic Testing Framework.** We develop mDLVerif [6], an automated dynamic testing framework based on differential analysis for real-world mDL reader apps, enabling systematic security assessment for mDL reader apps.
- **Evaluation of Real-world Implementations.** Our analysis of six mDL reader apps reveals that all of them contain at least one implementation flaw that could lead to security vulnerabilities.

2 Background

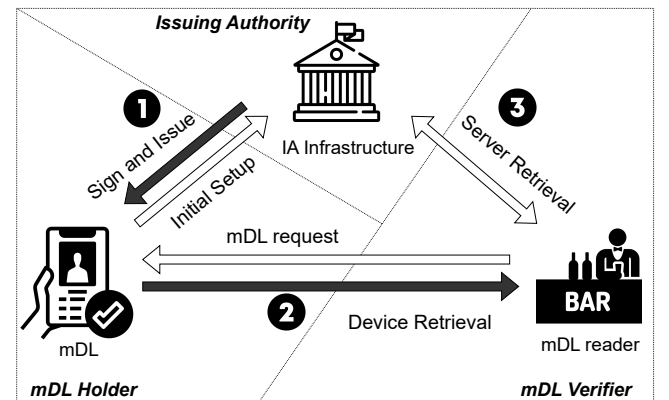


Figure 1: mDL interfaces

ISO/IEC 18013-5:2021, which we will refer to as the *mDL standard*, is a standard that outlines the specifications required to implement mobile documents (mdoc), including the Mobile Driving License (mDL). The mDL white paper [1] explains the functioning, expectations, and challenges associated with establishing an mDL ecosystem. Additionally, the mDL guideline [2] focuses on the interoperability and trustworthiness of mDLs from various issuing authorities, along with the importance of safeguarding privacy in their implementation. Please note that since ISO/IEC 18013-7:2025 had not yet been released when this work was conducted, we limit our scope to the requirements defined in the available ISO/IEC 18013-5:2021 standard.

mDL Fundamentals. The mDL standard enables the secure transfer and verification of an mDL. Figure 1 depicts the mDL usage process involving three entities: the Issuing Authority (IA), which issues and signs mDLs; the mDL holder, who stores and presents the mDL; and the verifier who uses an mDL reader to request and obtain the mDL. As per ISO/IEC 18013-5:2021, we consider only human verifiers. Unattended verification, described in ISO/IEC 18013-7:2025, is out of scope for our paper.

The mDL standard defines two methods for retrieving mDL: device retrieval and server retrieval. Device retrieval enables the mDL reader to communicate directly with the mDL without an Internet connection, relying instead on local trust establishment. This paper concentrates on the device retrieval method, as it presents a more challenging and less explored area of study.

Security Goals and Mechanisms. In terms of security, the mDL standard aims to ensure protections against common threats through four distinct security mechanisms (as shown in Table 1): Issuer data authentication prevents Forgery (BP1) by verifying data integrity against modifications. Mdoc authentication helps prevent Cloning (BP2) by confirming that the mDL is presented by its legitimate, authorized device, which is often combined with portrait authentication comparing the holder’s appearance to the mDL photo. Session encryption guards against Eavesdropping (BP3) by securing the communication channel during data exchange. Finally, mdoc reader authentication protects against Unauthorized Access (BP4) by allowing the mDL to verify the reader’s legitimacy before sharing data. These mechanisms provide enhanced protection for both mDL holders and verifiers.

It is important to note that mDL holder apps may implement additional protections such as Lock-Screen Knowledge Factors (e.g., PIN, password, pattern) or biometrics. These mechanisms are excluded from our analysis as they are not mandated by the ISO/IEC 18013-5:2021 standard. Furthermore, our threat model assumes that the mDL holder is malicious, hence local authentication is not relevant in this case.

Table 1: Security goals and mechanisms (simplified from the original table in the mDL standard [7])

Security Goal	Mechanisms
Protection against forgery (BP1)	Issuer data authentication
Protection against cloning (BP2)	mdoc authentication
Protection against eavesdropping (BP3)	Session encryption
Protection against unauthorized access (BP4)	mdoc reader authentication

Enhanced Privacy through Selective Disclosure. In contrast to traditional physical documents that require an all-or-nothing disclosure, an mDL offers a significant privacy advantage by allowing for selective information disclosure.

For instance, when verifying age at a liquor store, an mDL holder can choose to share only their age, unlike a physical license which exposes all the information on the document. This selective sharing aligns with the principle of minimal privilege, enhancing the holder’s privacy.

Cryptographic Proof for Authenticity. The mDL provides verifiers with cryptographic proof to ensure authenticity, enabling them to determine whether an mDL is genuine. This is achieved through a three-phase process: Initial Setup, Data Exchange, and mDL Verification. The detailed steps for this three-phase process can be viewed here [6]. We will illustrate how this is done by giving an example of how a person named Bob (B) applies for an mDL from an issuing authority (IA) and then uses it to provide data items to an mDL reader (R).

The initial setup of an mDL involves the IA securely retrieving and signing Bob’s personal data. This includes computing digests for each data item, creating a hash list, and signing it. The signature, hash list, and security-related information are then packaged into a Mobile Security Object (MSO), which Bob’s device retrieves.

During the Data Exchange phase, Bob’s mDL communicates with a reader using encrypted channels. The reader requests minimal necessary information for authentication, and Bob can choose to withhold certain data items.

Finally, in the mDL Verification phase, the reader authenticates the issuer’s data, the mDL document, and the reader itself to prevent Forgery, Cloning, and Unauthorized Access. It also performs portrait authentication to confirm the holder’s identity. Successful completion of these steps allows the verifier to authorize Bob for the requested task. A data item of interest in this phase is docType. This data item is present in both the MSO and the data item set, indicating the type of document, such as a driving license (“org.iso.18013.5.1.mDL”). It connects the MSO and data items, allowing an authenticated MSO to verify the data items.

3 Identifying Implementation Requirements

The security of the mDL standard protocol, in practice, depends on its precise implementation. Hence, to evaluate its security in real-world applications, we extract critical implementation requirements (i.e., security checks) from the specification that developers must follow when developing mDL reader apps.

Threat Model. We consider the mDL holder as the potential adversary, focusing on threats where the document holder attempts to deceive the verifier. Attacks from compromised Issuing Authorities (IAs) are considered out of scope as they would undermine the trust foundation of the entire system.

Please note that since we assume the existence of a malicious mDL holder, the use of lockscreen or biometrics is not relevant in our threat model. We assume that the person presenting the mDL has passed the authentication mechanisms implemented on the device holding the mDL. This might have been either because the user owns the device or they maliciously gained access to the device (e.g., by obtaining the owner’s password to unlock the screen). In

other words, in our threat model, the person presenting the mDL (i.e., the mDL presenter) effectively becomes synonymous with the person holding the mDL (i.e., the mDL holder).

Focus and Scope. Within the ISO/IEC 18013-5:2021 standard, we concentrate on mDL-specific requirements related to document verification, excluding transmission layer (e.g., Bluetooth, NFC) security aspects, which are governed by other protocols with their own security considerations. Furthermore, the mDL standard describes two mDL retrieval methods: device and server retrieval. This paper focuses on the device retrieval method, which allows for trust establishment between the mDL and the mDL reader even without an Internet connection, presenting a more challenging and less explored area of study. Server-based verification methods that utilize online connectivity to perform traditional checks are excluded from our analysis.

Approach. We first manually studied the mDL specifications to identify implementation requirements for reader apps. In parallel, we used LLMs to independently extract implementation requirements from the same specifications. Both approaches converged on the same set of seven critical requirements, providing cross-validation of our findings and strengthening the comprehensiveness of our analysis. Through this process, we identified seven critical implementation requirements (i.e., security checks). When these requirements, denoted with ‘C’, are not properly implemented, they become security pitfalls that compromise the security of the mDL verification process and enable attacks as demonstrated by M'_{mDL} . These security checks are later used as the basis for our dynamic testing framework mDLVerif to evaluate real-world mDL reader apps (Section 7).

Table 5 in Appendix A summarizes the seven critical implementation requirements for mDL reader apps, extracted from the mDL standard specification with their corresponding page and quotes. Each requirement addresses a specific security concern, ranging from authentication mechanisms (C1–C3) to data integrity verification (C4–C5) and validity checks (C6–C7). When these requirements are not properly implemented, they become security pitfalls, enabling various attacks. On the contrary, as we will show in Section 5 using formal verification, the correct implementation of the C1–C7 requirements guarantees that a reader app satisfies the security requirements mentioned in Section 2 and Section 3.

We discuss how these pitfalls cause vulnerabilities and how the presence of these mistakes in an app’s implementation can be exploited in Section 5. The mDL standard typically automates procedures through algorithms, but verification of an mDL holder’s picture often requires manual checking. This process, often just an eye test, is prone to errors, like failing to distinguish an impersonator under poor lighting. Such vulnerabilities, leading to identity theft and other issues, are well-documented in various studies, including [9][32][31][2], with human error in facial comparison occurring about 2.47% of the time [24]. This highlights the need to evaluate the mDL standard’s effectiveness in situations where security features are compromised with or without human error. Therefore, we verify 7 implementation requirements in two settings: one involving human error in portrait authentication, and the other without it.

4 Formal Model Design and Implementation

In this section, we identify the two main challenges in modeling mDL and detail the strategies and techniques we implemented to effectively address these technical challenges. Then, we outline the process of modeling the three mDL phases in ProVerif. Lastly, we explain how we validate security properties and extend the formal model to verify implementation pitfalls.

4.1 Design Challenges and Solutions

Challenge 1: Managing Variants. One significant challenge is handling the 15 model variations (1 reference, 14 incorrect implementations) across 15 queries (11 real-world usage scenarios and 4 basic security properties.) The 14 incorrect implementations arise from modeling each of the 7 identified pitfalls in two distinct ways: with successful portrait authentication and without it. This results in 225 cases to verify across our formal model. The obstacles in this aspect are twofold. First, creating a single model for all variants is impractical due to differing threat models, the complexity of a unified model, and the exponential increase in ProVerif verification time for more variants. Second, each variant involves approximately 500 lines of code. Manual implementation and maintenance of such a large volume of largely duplicate code are unfeasible and prone to errors.

Challenge 2: Future-Proofing. As mDL gains popularity, the emergence of new usage scenarios and implementation errors is inevitable. The current pitfalls identified are not exhaustive and may expand over time. Thus, our formal model requires regular updates to remain effective. Manually crafting a new model for each update is time-consuming and prone to errors, especially if extensive modifications are necessary.

Solutions. In response to the identified challenges, we design a modular framework using a template engine in conjunction with ProVerif, focusing on flexibility and extensibility to cater to diverse verification scenarios. This framework features a central “main module,” complemented by reusable “component modules” and adaptable “scenario modules”. Integral to this design are two functionalities: “module import” and “module specialization.” The module import capability enables seamless integration and reuse of code across different modules, allowing for the efficient assembly of complex models from smaller units. Module specialization, on the other hand, utilizes templating for the customization of modules based on user parameters.

Utilizing this framework eases the verification of additional usage scenarios. It saves time by eliminating the need to understand mDL internally and reduces coding efforts via code reuse. For instance, a generated model for a specific scenario has approximately 500 lines of code on average, but we verified Scenario 3 just by writing 60 new lines. Details on how to use this verification framework can be found in [6].

4.2 Verified Properties

In our study, we translate the four security objectives of the mDL standard (see Section 2) into corresponding basic security properties. Then, based on the mDL white paper [1], we evaluate mDL’s security in 11 real-world use cases, as outlined in Section 4.3.

Furthermore, we systematically verify each real-world usage scenario against a set of seven implementation requirements (i.e., security checks) for mDL reader apps. These requirements, detailed in Section 3, were identified through both manual analysis and extraction using LLMs. When improperly implemented, these requirements become security pitfalls that compromise the verification process. Although the specification clearly outlines these requirements, the security implications of incorrect implementation are not always apparent.

To understand how attacks can be executed, we verify basic properties and real-world scenarios with incorrect implementations to generate step-by-step attack traces. These traces illustrate how attacks unfold when security checks are missing, enabling us to precisely construct malicious mDL documents for our automated analysis implementation described in Section 7.

4.3 Real-world Usage Scenarios of mDL

To evaluate the security of the mDL standard in concrete, real-world examples, we verify 11 usage scenarios. These scenarios originate from the white paper written by the STA [1]. All the verified scenarios can be found in Table 4, located in Appendix A.

In each scenario, the verifier might ask different questions, which we will refer to as “business questions.” By asking a business question, a real-world problem is being addressed, and the answer will be used as a basis for verification or authentication in a real-world scenario. We assume the attacker does not possess the necessary privileges or identities to succeed in the corresponding verification or authentication. Then, we verify if the attacker can circumvent business questions and proceed without providing expected answers.

4.4 Modeling the mDL Protocol

In our formal model, we use three ProVerif processes (holder, reader, and issuer) to represent the mDL transaction participants. Each process implements steps with component modules that are either “replaceable” (interchangeable for different scenarios) or “specialized” (reused with different parameters). Figure 5 in Appendix B illustrates the ten steps across three phases of the protocol implementation.

Initial Setup Phase (Steps 1-5). The Initial Setup phase establishes a secure environment by creating a protected communication channel between the issuer and mDL processes. To model the transition from a secure setup to a potentially compromised environment, we split both the issuer process and its communication channel into two separate parts.

Steps 2 and 3 are implemented as replaceable modules, allowing the issuer to impose scenario-specific restrictions and issue only essential mDL items for each scenario. In Step 4, the issuer emits an `ia_sign` event when signing each mDL item, indicating issuance. This event is later used to detect potential forgery attacks.

Data Exchange Phase (Steps 6-8). During this phase, communications between the mDL and reader processes occur over a public channel, with session encryption implemented as specified in the mDL standard. In Step 6, the mDL authenticates the reader process by verifying its signature, triggering a `reader_authenticated`

event if successful. This authentication prevents unauthorized access attacks.

In Step 7, after reader verification, the mDL process authenticates its responses by emitting a `device_sign` event for each item, demonstrating the device’s authorization to release them. Step 8 completes the exchange as the holder provides the requested mDL data, along with security elements like `issuer_auth` and `device_auth` to ensure data integrity.

mDL Verification Phase (Steps 9-10). The final phase involves the reader process verifying received items through required inspections. Successful verification triggers `issuer_authenticated` and `device_authenticated` events for each data item, with an `item_accept` event indicating complete validation. The mDL is accepted when all requested items are successfully verified.

In Step 10, the reader verifies the business-specific requirements for various scenarios using a replaceable module. A `business_accept` event signals successful verification of a business question, indicating the mDL meets all requirements for that particular use case.

4.5 Querying the Formal Model

Using M_{mDL} , we verify security properties in both basic and real-world usage scenarios through queries. The basic scenarios we query include Eavesdropping, Forgery, Cloning, and Unauthorized Access. Additionally, we query 11 real-world usage scenarios, detailed in Table 4 in Appendix A.

To verify mDL integrity against forgery (BP1), we check the correspondence assertion between `issuer_authenticated` and `ia_sign` events. For mDL authenticity against cloning (BP2), we assess the correspondence assertion linking `item_accept` in Step 9 with `ia_sign` in Step 4, ensuring presented items were issued by the IA. To protect against eavesdropping (BP3), we set mDL items as private and query their reachability. For unauthorized access (BP4), we use a trusted certificate for the reader process and compare the `reader_authenticated` event with `reader_sign` in Step 6 to authenticate the reader process, ensuring all accepted mDL requests were signed by a trusted reader.

4.6 Extending the Formal Model

To generate step-by-step attack traces for each of the implementation pitfalls identified in Section 3, we extended M_{mDL} to M'_{mDL} and simulated implementation mistakes in four basic properties and 11 real-world usage scenarios within the model. We achieved this by running the model multiple times for each property and scenario. For each run, we simulated one implementation mistake to generate the specific attack trace resulting from that error in the given scenario or basic property. These detailed attack traces were crucial for implementing our automated app analysis tool described in Section 7.3.

Specifically, in addition to querying four basic properties, we examined 11 real-world scenarios to see if an attacker could trigger the `business_accept` event without the required privileges or identity. This event should only occur if security is breached. For instance, in scenario 4, customers must be over 21 to buy age-restricted items. We altered Step 3 to prevent issuing an mDL indicating the attacker is over 21, and in Step 10, we ensured the reader process verifies

age. We then queried whether the attacker could still trigger the `business_accept` event, which would compromise the security of the business question.

Furthermore, we also model the special case of human error in portrait authentication (C7) as mentioned in Section 3. We verify each implementation requirement in two settings: one involving human error in portrait authentication, and another without it. This allows us to check which properties and scenarios are ultimately reliant on portrait authentication for their security and which of them can guarantee their security properties even without accurate portrait authentication.

5 Discussion of the Formal Verification Results

The formal verification of M_{mDL} showed that when the protocol is implemented correctly, all four basic security goals specified by mDL standard and the security properties for all 11 scenarios are guaranteed.

Specifically, when running M_{mDL} in ProVerif, the absence of attack traces formally guarantees that the implementation satisfies all of the modeled security requirements. In particular, the fact that ProVerif does not produce any attack trace formally proves that the attacker has no way to forge data items, clone an mDL, eavesdrop on communications, or gain unauthorized access. As a result, all four basic security properties—as well as the security requirements defined across the 11 real-world usage scenarios (Section 4.3)—are simultaneously satisfied. In other words, the absence of attack traces produced by our formal analysis directly indicates that none of the adversarial behaviors we modeled can lead to a violation of any of these properties. Overall, these results demonstrate that an mDL reader app that correctly implements all seven security checks (C1–C7) is provably secure under the adversarial holder threat model described in Section 2 and Section 3. This outcome aligns with the mDL standard security objectives.

However, M'_{mDL} showed that the implementation pitfalls we identified can cause vulnerabilities that not only allow an attacker to bypass the security of certain scenarios but also allow an attacker to bypass the two out of the four basic security properties guaranteed by the mDL standard. Table 2 shows the complete results of both M_{mDL} (REF) and M'_{mDL} , assuming both accurate and inaccurate portrait authentication.

Importantly, our formal verification not only identifies when attacks are possible but also provides detailed step-by-step attack traces that illustrate how these vulnerabilities can be exploited. These attack traces were used in our construction of malicious mDL documents for the automated analysis implementation described in Section 7, allowing us to systematically test real-world mDL reader apps against these security pitfalls.

Our formal verification identifies both trivial and complex attacks using M'_{mDL} . For example, it effectively highlights attacks, such as cloning due to lacking mdoc authentication (C1) and forgery due to no verification of the certificate chain (C2). These vulnerabilities, along with the absence of critical security checks like MSO signature authentication (C3) and item digest checks (C4), severely compromise the security of the mDL protocol.

Beyond these straightforward issues, M'_{mDL} also reveals more complex attacks like the “docType Forgery.” It shows that without

Table 2: Verification of each Basic Property (BP) and Scenario (SC) for M_{mDL} and M'_{mDL} . The REF column represents the results of M_{mDL} . ✓: All security guarantees are achieved with and without accurate portrait authentication. ●: Security guarantees are only achieved with accurate portrait authentication. ✗: Security guarantees cannot be achieved.

BP/SC	C1	C2	C3	C4	C5	C6	C7	REF
BP1	✓	✗	✗	✗	✗	✗	✓	✓
BP2	✗	✗	✗	✓	✓	✓	✓	✓
BP3	✓	✓	✓	✓	✓	✓	✓	✓
BP4	✓	✓	✓	✓	✓	✓	✓	✓
SC1	✓	✗	✗	✗	✓	✗	✓	✓
SC2	✓	✗	✗	✗	✓	✗	✓	✓
SC3	✓	✗	✗	✗	✓	✗	✓	✓
SC4	●	✗	✗	✗	✓	✗	✓	✓
SC5	✓	✗	✗	✗	✓	✗	✓	✓
SC6	✓	✗	✗	✗	✓	✗	✓	✓
SC7	✓	✗	✗	✗	✓	✗	✓	✓
SC8	✓	✗	✗	✗	✓	✗	✓	✓
SC9	✓	✗	✗	✗	✓	✗	✓	✓
SC10	●	✗	✗	✗	✓	✗	✓	✓
SC11	✓	✗	✗	✗	✓	✗	✓	✓

checking the docType (C5), an essential element linking the MSO to the data items, there is a violation of the basic property to protect against forgery (BP1). In the absence of a docType check, a reader is unable to ascertain whether a valid MSO is associated with what appears to be a valid set of data items. This pitfall allows attackers to exploit the business question by presenting a document with mismatched docType and MSO, such as presenting a car registration as a driver’s license.

Another insight from M'_{mDL} highlights a critical vulnerability: when the reader application fails to verify the MSO signature’s validity (C6). The importance of MSO’s validity might be overshadowed by the driver’s license’s own ‘validity,’ represented by the expiry_date of driving privileges. Intuitively, overlooking the MSO’s expiry date might seem inconsequential. However, ProVerif shows that this pitfall violates the basic property to protect against forgery (BP1). Additionally, it allows an attacker to use expired mDL information to bypass security in all 11 scenarios, such as accessing a secure area after access has been revoked. This can occur if a malicious mDL owner stops their mDL from updating, retaining access to the secure area despite the IA revoking it. Without checks for ValidityInfo, the mDL owner can continue using the expired mDL to access the area as long as it remains not updated.

We note that portrait verification (C7) is the only pitfall we identified that does not bypass the security guarantees of the mDL standard. However, as can be seen, by Table 2, there are 2 cases where inaccurate portrait authentication in combination with another pitfall can result in a bypass of the security guarantees. Specifically, in the absence of accurate portrait authentication, if mdoc authentication is also missing, an attacker can use the mDL of a legitimate owner to purchase alcohol (SC4) or open bank accounts in the name of the original holder (SC10).

6 mDLVerif Design

In this section, we discuss our tool designed to systematically automate the analysis of mDL reader apps on the market. We begin by explaining the key technical challenges involved, and then present our approach, mDLVerif, which is designed to overcome these challenges. Building on the formal model introduced in Section 4.6, we demonstrate how M'_{mDL} provides a systematic framework that addresses one of the technical challenges. Finally, we explain the implementation details of mDLVerif and how it comprehensively verifies apps' implementation of the security verification requirements outlined in Section 3.

Technical Challenges. Preliminary manual reverse engineering of mDL reader apps revealed several critical technical challenges in conducting automated analysis of mDL apps. We summarize these challenges as follows:

CH1: Heterogeneous, Obfuscated Implementations: Many mDL reader apps employ sophisticated code obfuscation techniques while utilizing different custom libraries with no standardized implementation of the mDL protocol. Some apps even dynamically download and execute mDL-related code at runtime. This combination of heterogeneity and anti-analysis methods makes it exceptionally challenging to perform effective static analysis, as the code to be analyzed may not be readily available and implementation patterns vary significantly across applications.

CH2: Systematic Test Cases Generation: The mDL protocol complexity, with numerous usage scenarios and multi-step procedures, makes manual identification of all attack vectors challenging. Without a systematic approach to both enumerate potential vulnerabilities and identify which parts of mDL's intricate data structures are security-critical, testing can become inefficient, potentially missing critical edge cases.

CH3: Non-standardized security indicators: mDL reader apps display verification results through diverse visual feedback mechanisms, such as checkmarks, colored borders, or status messages, that are easily interpretable by humans but create significant challenges for automated security analysis tools. The lack of standardization in how security outcomes are presented makes it difficult to programmatically determine whether an app correctly identifies and reports security issues.

Solution. To address CH1, we adopt a dynamic-analysis-based approach. By analyzing apps during runtime execution, we can observe actual behavior after code obfuscation has been processed and dynamically loaded modules become available. Furthermore, despite the heterogeneous implementation landscape of mDL, the interaction protocol between holder and reader apps remains standardized as specified in the mDL standard. This standardization allows mDLVerif to use a single comprehensive mDL holder app to interact with any compliant reader app, providing a unified approach to analyze diverse implementations while ensuring all security-relevant code is accessible during assessment.

To address CH2, we leverage our formal model M'_{mDL} from Section 4.6, which covers four basic properties, 11 real-world usage scenarios, and seven implementation requirements. Specifically, we use M'_{mDL} to generate step-by-step attack traces that serve as blueprints for our test cases. This method helps us identify which parts of mDL's data structures are security-critical and which paths

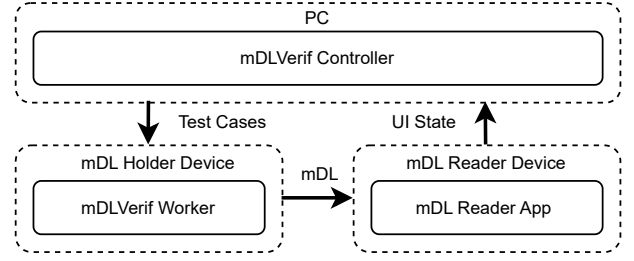


Figure 2: Overview of the mDLVerif architecture

need verification, providing consistent coverage across diverse implementations. Rather than relying on intuition, our formal approach ensures systematic exploration of potential vulnerabilities in the complex mDL protocol.

To address CH3, we employ differential analysis by first establishing a baseline with compliant mDL documents, then comparing against UI states produced by malicious ones. Visual differences, such as green vs. red indicators, or presence vs. absence of warnings, suggest proper security check implementation, while identical states reveal vulnerabilities where non-compliant documents fail to trigger distinct visual feedback. We chose differential analysis of the entire UI state because mDL reader apps have no standardized way of reporting verification outcomes—some show explicit success indicators, others display nothing on success but only errors on failure, making it impossible to identify success indicators, and future apps may use entirely different mechanisms. Our approach leverages relative relationships between visual states rather than predefined rules, enabling effective detection across diverse implementations.

7 mDLVerif Implementation

In this section, we first present the architecture of mDLVerif, as well as an overview of the dynamic analysis procedure in Section 7.1. Then, in Section 7.2, we explain in detail how mDLVerif constructs a baseline. Moreover, we explain how mDLVerif uses the attack traces generated from formal verification to construct malicious mDL documents in Section 7.3. Finally, in Section 7.4, we discuss two main types of problems that cause false negatives in differential analysis. We also explain how mDLVerif employs a novel approach to systematically address these problems, mitigating false negatives, and ensuring that the differential analysis is accurate and produces reliable results.

7.1 Overview of mDLVerif

As illustrated in Figure 2, the mDLVerif framework operates across three devices: a PC, an mDL holder device, and an mDL reader device. The framework consists of two primary components: a controller and a worker app. The controller, running on the PC, orchestrates the entire mDL verification process by coordinating interactions between the devices. It generates mDL data and transmits it to the worker app running on the holder device. Upon receiving this data, the worker app constructs an mDL, then transmits it to the reader device. The reader device, which runs the target mDL reader app under test, processes the received mDL document. Then,

the controller captures the UI state from the reader device and uses it for the differential analysis process. To automate interaction with reader apps, mDLVerif uses Frida [4] to record touch gestures and navigation patterns during the first test execution. These recorded interactions are then replayed in subsequent test runs.

The dynamic analysis procedure consists of three phases: First, mDLVerif transmits a fully compliant mDL to the reader app and captures the baseline UI state. Second, it generates a non-compliant mDL and sends it to the reader app, subsequently capturing the resulting UI states after processing. Third, it performs differential analysis between the baseline UI state and the non-compliant UI states, identifying visual differences that enable human verifiers to distinguish between compliant and non-compliant mDL documents.

During the first phase, even when mDLVerif constructs a fully compliant mDL, the reader app may reject it because it does not trust the testing certificate used by mDLVerif to sign the document. To address this issue, we adopt dynamic code instrumentation to establish the trust anchor, ensuring the reader app properly authenticates our fully compliant mDL document, as detailed in Section 7.2.

Additionally, even with trusted certificates, the reader app might not support all required verification features, potentially causing crashes or exceptions. To ensure valid baselines for our analysis, we capture and examine screenshots during baseline construction to verify successful mDL authentication. More details are provided in Section 7.2.

For the third phase to highlight vulnerabilities, mDLVerif requires exactly identical UI states between compliant and non-compliant mDLs. However, generating perfectly identical UI states is challenging due to the versatile and fragile nature of the dynamic testing environment, where factors such as timing variations can lead to false negatives. To mitigate this issue, we systematically identify and collect expected differences between UI states, which are then excluded from differential analysis. More details about this approach are provided in Section 7.4.

7.2 Constructing the Baseline

Constructing a proper baseline requires additional technical measures other than just sending a fully compliant mDL to the reader app. In this section, we explain the implementation details of how mDLVerif constructs a baseline.

Injecting Certificates. To perform the differential analysis, as shown in Figure 3, mDLVerif first establishes a baseline by sending a fully compliant mDL document to the reader app and capturing the resulting UI state. This baseline corresponds to the behavior that the mDL reader app has when provided with a compliant mDL.

The fully compliant mDL is generated using our own self-signed certificate. However, as detailed in Section 2, the mDL standard requires that reader apps must verify the trust chain of an mDL and reject documents signed using untrusted certificates. Since our testing framework uses testing certificates that are outside the reader apps' trusted stores, the reader app may reject our self-created compliant mDL. To address this, we implement a dynamic certificate replacement mechanism that operates during runtime. This mechanism intercepts the certificate validation process and modifies it to make it accept mDLs signed using our

testing certificate. More specifically, our implementation leverages Frida [4] to dynamically instrument the tested app, hooking multiple entry points commonly used for certificate initialization, such as `CertificateFactory.generateCertificate`. By exploiting these hooks, mDLVerif then replaces, at runtime, one of the trusted production certificates trusted by the tested app with the testing certificate that mDLVerif uses to sign the baseline mDL.

Handling Invalid Baseline. Furthermore, even if the reader app trusts the certificate used by mDLVerif, the UI states may not accurately represent the actual UI state when the reader app authenticates a compliant mDL. This discrepancy can occur because the reader app might not support all the mDL verification features required by our test cases. In such scenarios, the reader app might crash or throw an exception when processing the mDL provided by mDLVerif. If the baseline is not valid, subsequent differential analysis will not produce reliable results. To address this challenge, we preserve all screenshots captured during baseline construction and later examine these screenshots to verify whether the baseline mDL actually demonstrated successful authentication.

A special case arises when the mDL reader app does not support certain features of the mDL verification which are required by the test cases. In this case, the baseline mDL no longer represents “what it looks like when the reader authenticates a compliant mDL,” and therefore cannot be used for subsequent differential analysis.

To address this challenge, we preserve all screenshots captured during baseline construction and later examine these screenshots to verify whether the baseline mDL was actually authenticated. We implement a dual verification approach: we manually inspect these screenshots while simultaneously employing a Vision Language Model (VLM) to verify the screenshots as an additional validation measure. If a screenshot indicates that the mDL was not properly authenticated, we mark the corresponding test case as “Not Applicable”.

7.3 Constructing Malicious mDLs

We construct malicious mDL documents by following attack traces generated from our formal verification. Table 2 indicates which attacks are possible when specific security checks are missing. We manually translate attack traces into test cases in mDLVerif. mDLVerif then automatically generates a malicious mDL, signs it, and sends it to reader apps to attempt to replay the attack. By sending these malicious mDLs to reader apps and observing whether they can distinguish them from compliant mDLs, we can determine if the necessary security checks are implemented.

For example, Table 2 shows that the “Protection against Forgery” (BP1) cannot be guaranteed if the reader app fails to check the “docType” of the mDL (see Section 5). Specifically, our revised formal model (M'_{mDL}) shows the following trace (simplified) for the attack:

- (1) *Holder* requests *Issuer* action regarding legitimate *DocType B*, providing chosen *any_data*. This interaction gets the *Issuer* involved with the attacker's data.
- (2) *Issuer* returns a data structure containing ‘Hash(*any_data*)’, with a valid ‘*Issuer Signature*’ created for *DocType B*. *Holder* now has *Issuer*'s signature related to their data.
- (3) *Holder* initiates presentation to *Reader*, intending to misuse the obtained signature.

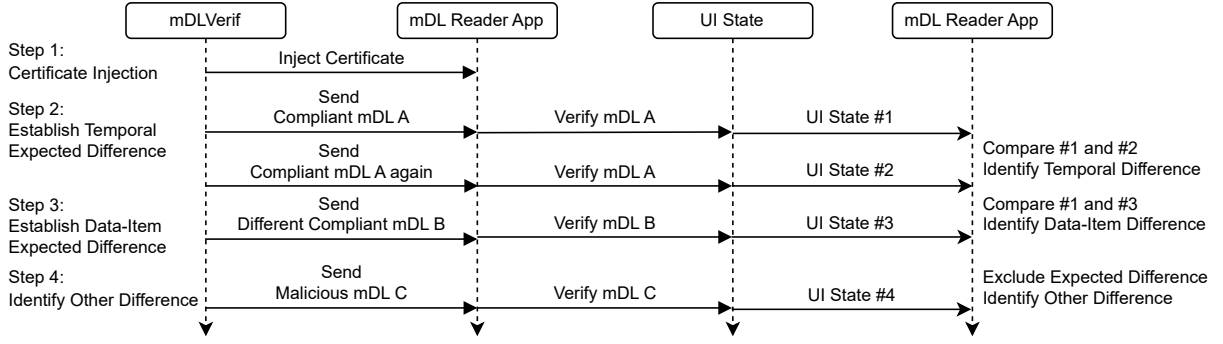


Figure 3: Overview of the Differential Analysis Approach

- (4) *Holder* presents the ‘*Issuer Signature*’ and data structure (from step 2) but falsely claims the document type is *DocType A*. *Holder* also reveals *any_data* and adds its own valid device signature.
- (5) *Reader* validates the ‘*Issuer Signature*’ is authentic and the hash matches the disclosed *any_data*. *Holder*’s device signature is also valid.
- (6) *Reader* critically fails to check if the ‘*Issuer Signature*’’s original context (*DocType B*) matches the *Holder*’s declared context (*DocType A*).
- (7) Due to the missed context check, *Reader* incorrectly accepts *any_data* as *Issuer*-authenticated under *DocType A*. The attack succeeds.

To implement the “docType Forgery” attack in mDLVerif, we construct a malicious mDL by following steps (1) through (5) from the attack trace. This allows us to precisely replicate the vulnerability conditions. We apply the same methodology to construct malicious mDLs for other security vulnerabilities, using their specific attack traces.

ProVerif produces one attack trace per scenario. We turn each trace into multiple concrete test cases using a two-phase process designed to maximize coverage. Specifically, first, we extract the general attack pattern from the formal trace. This pattern captures the essential exploit flow enabled by a specific attack. Next, we identify the points in that pattern where the attacker has arbitrary control over the mDL data and where modifying such data does not disrupt the overall exploit logic. By systematically enumerating the possible manipulations at these controllable steps, we expand each ProVerif trace into multiple test cases, each representing a distinct malicious mDL variant that simulates a plausible incorrect implementation.

ProVerif’s attack traces do not explicitly show what enabled the attack, but rather they only demonstrate specific issues. For this reason, to identify and extract the general attack pattern, we analyze the trace corresponding to each violated query step by step. For instance, suppose that we assert that an attacker should never be able to pass a specific check (e.g., age verification) when they have only been issued an mDL that should fail that check (e.g., an mDL indicating an insufficient age). If ProVerif nevertheless reports that the query is satisfiable, we carefully, manually review the trace step by step, comparing each operation against the standard

mDL verification procedure. Any deviations from the expected behavior (e.g., tampering with an existing field, substituting a field from another mDL, etc.) are marked as non-standard actions. These actions are identified through manual inspection informed by the standard mDL verification workflow, allowing deviations in the trace to be recognized as abnormal operations.

For example, if a trace reveals that a general attack pattern is possible due to a reader app failing to properly validate the age attribute, we first identify this pattern from the formal trace generated by ProVerif. Then, we identify that the attacker has arbitrary control over the age attribute and that any modification to this attribute would not break the underlying attack pattern. As a result, we generate multiple malicious mDLs (i.e., multiple “test case” variants), including mDLs in which (a) we omit the age attribute entirely, (b) we provide an invalid age value that violates data type constraints, and (c) we provide an age value that fails application-specific business logic checks.

While our approach shares elements with fuzzing, it offers two key advantages: it generates test cases efficiently and acts as a comprehensive black-box oracle (Section 6). Regular fuzzing might make reader apps misbehave, but would not systematically mutate the complex mDL data structure to exploit vulnerabilities as efficiently as mDLVerif does, and would struggle to identify the subtle, logic-based flaws that mDLVerif can find. Conversely, our formal model provides step-by-step attack traces that act as blueprints, telling mDLVerif exactly what security-critical data to modify (e.g., the docType field). Our approach does not rely on validating intermediate results based on the app’s internal checks. This allows mDLVerif to flag vulnerabilities even when an app’s internal checks pass but the result is not visually communicated to the human verifier (e.g., the CredenceID case in Section 8.3).

7.4 Mitigating False Negatives

During our work, we identified two main types of problems that cause false negatives in differential analysis: 1. Temporal variations in UI rendering, and 2. Variations in data items such as name, date of birth, or document numbers. In this section, we explain how mDLVerif systematically identifies and excludes these expected differences from the differential analysis.

In order to precisely identify the element that causes the difference, mDLVerif utilizes UI Automator [8] to capture the UI state

in the form of an XML tree. Then, we identify the exact node from the XML tree that causes the difference between the two UI states.

Identifying Temporal Variations. Even when sending the same mDL document twice, we identified that the UI states can be different for certain mDL reader apps. This is usually due to the UI rendering time-dependent elements, such as a countdown timer, timestamp, or other time-dependent elements. For example, the baseline UI state could be “Authentication Success, 10 seconds left”, while the non-compliant UI state could be “Authentication Success, 5 seconds left”. In this case, this app is not correctly implemented due to the fact that it shows “Authentication Success” for both compliant and non-compliant mDLs. However, the time-dependent element causes the two UI states to be different, which leads to a false negative.

To address this, mDLVerif sends identical mDL documents at two distinct points in time and compares the resulting UI states to detect inconsistencies. We identify UI elements causing inconsistencies and add them to the list of expected differences. As shown in Step 2 of Figure 3, mDLVerif sends the same compliant mDL document (mDL A) twice at different times to collect UI state #1 and #2. By comparing UI states #1 and #2, mDLVerif can account for temporal differences that naturally occur between identical mDLs.

Identifying Data-item Variations. Another type of false negatives is caused by variations in data items such as name, date of birth, or document numbers that naturally vary between different valid mDLs. For example, when verifying an incorrectly implemented mDL reader app, mDLVerif could present a compliant mDL with the name “Alice” and collect the baseline UI state as “Authentication Success”. Then, when an attacker presents a non-compliant mDL, with the name as “Bob”, since the reader app is incorrectly implemented, the UI state would be “Authentication Success”. In this case, the two UI states are different solely because of the data item, which is not relevant to the security verification. In fact, this app should be flagged as “Incorrectly Implemented” due to the fact that it has the same UI state “Authentication Success” for both compliant and non-compliant mDLs.

To address this issue, we identify the exact data item that causes the difference between the two UI states, and we add it to the list of expected differences. As shown in Step 3 of Figure 3, mDLVerif sends a different but still compliant mDL document (mDL B) to generate UI state #3. By comparing UI states #1 and #3, mDLVerif can identify and account for data-item variations that naturally occur between different valid mDLs.

8 Analyzing Real-world mDL Reader Apps

In this section, we utilize mDLVerif (see Section 7) to conduct a comprehensive analysis of real-world mDL reader apps. We begin by detailing our methodology for identifying mDL reader apps in the marketplace. Subsequently, we present a summary of the implementation vulnerabilities uncovered during our investigation. Finally, we present one representative case study, with additional case studies provided in Appendix D.

8.1 Identifying mDL Apps

To identify reader apps that potentially implement the mDL standard, we conducted static and manual analysis and additionally

used the Gemini Deep Research tool [5]. More details are provided in Appendix C.

By combining the results from both our static/manual analysis and the Gemini Deep Research tool (and removing duplicates), we identified a total of eight unique mDL reader apps. However, two of these apps could not be analyzed with mDLVerif, as we were unable to successfully inject certificates into them, preventing us from generating a baseline for differential analysis. As a result, we proceeded to analyze the six remaining mDL reader apps.

8.2 Identified Security Issues

In this section, we will analyze and discuss four types of common issues that we identified in various mDL reader apps. As shown in Table 3, these issues exist in more than one reader app and highlight the risks that exist when specifications are not strictly adhered to during implementation. To illustrate our findings, we also demonstrate attacks based on the identified security issues [3]. We will also discuss the potential consequences of these vulnerabilities.

As detailed in Section 7.2, mDLVerif establishes a baseline by transmitting a fully compliant mDL to the reader app. When an app fails to authenticate this baseline due to crashes, unsupported features, or other issues, differential analysis becomes impossible since it requires a valid baseline as a starting point. In such cases, we categorize the result as “N/A”.

Optional Portrait. The first security issue we identified concerns the implementation of the portrait as an optional data item. Even though selective disclosure is part of the mDL standard’s design, the mDL standard treats the portrait differently. In fact, the standard explicitly mandates that if *any* data element is shared with a reader, the portrait (as the primary means to verify the presenter is the holder) must also be returned if it was requested. As stated in the standard [23]:

If any data element is returned to the mDL reader, then the information necessary to verify that the person presenting the mDL is the mDL holder shall be returned to the mDL reader if that information is requested.

– BS ISO/IEC 18013-5:2021, section 7.2.1, page 20

Therefore, if an mDL holder does not return a requested portrait, this indicates non-compliance with the standard on the part of the holder, and the verifier cannot authenticate the user without relying on secondary mechanisms such as attested biometrics. Since our threat model explicitly focuses on malicious mDL holders, these alternative authentication mechanisms are considered out of scope for our work. Consequently, while the holder’s failure to provide a portrait is not itself an explicit violation of the standard from the reader app’s perspective, we expect mDL reader applications to explicitly signal an error or warning to the verifier when a requested portrait is not returned, because, as explained, authentication is not possible in our considered scenarios without a portrait. We therefore consider reader apps that silently proceed without issuing such an error or warning to be deviating from best practices.

Expired MSO. The second issue we found in libraries is missing the validity (expiration date) check for MSO. mDLs have two types of validity: the driver’s license validity and the MSO validity. Reader apps must verify both according to specifications. As stated in the standard [23]:

Table 3: The Analysis Results of mDL Reader Apps

Installation	Google N/A	MDOT 100+	CredenceID 500+	GET 5k+	Scytale 100+	Idemia 500k+
C1. mdoc Authentication	✓	✓	Acknowledged	✓	✓	✓
C2. Trust Anchor	✓	✓	Acknowledged	✓	✓	✓
C3. MSO signature	✓	✓	Acknowledged	✓	✓	✓
C4. Items digests	✓	✓	Acknowledged	✓	✓	✓
C5. docType match	N/A	N/A	N/A	Acknowledged	Reported	N/A
C6. Validity Check	Fixed	✓	N/A	✓	✓	✓
C7. Holder Picture	Fixed	Reported	✓	Acknowledged	Reported	Reported

Note: "N/A" indicates that baseline construction for differential analysis failed (e.g., due to an app crash or an unsupported feature). The check mark (✓) signifies that a *difference* was detected in the UI state when comparing the results from verifying compliant versus non-compliant mDLs. For issues we found where apps showed no difference in the comparison (indicating a vulnerability), we marked them as "Reported". If we received acknowledgment from the developer for a reported issue, we marked it as "Acknowledged". For apps that fixed the issue after our report, we marked them as "Fixed".

The `validUntil` element determines the validity period of the MSO and therefore, the mdoc cannot be validated after this date.

– BS ISO/IEC 18013-5:2021, section 9.1.2.4, page 51

Failing to validate `ValidityInfo` for MSO can lead to security risks, as this mechanism ensures periodic reevaluation of an mDL holder's data. An expired MSO overlooked by an app could allow Unauthorized Access and display outdated information, leading to incorrect verification. This could enable someone to falsely appear younger, potentially gaining unwarranted benefits like insurance discounts. We identified this issue in one of the six examined mDL reader apps.

docType Forgery. Another common issue we discovered is the absence of checking `docType` of MSO. The mDL includes an MSO, utilized by the mDL reader to verify the data's integrity. Both the mDL and MSO possess a `docType` attribute, linking the MSO to the mDL data items. In fact, the mDL standard [23] mandates that mDL reader apps should ensure that the `docType` of the provided mDL matches the `docType` of the MSO:

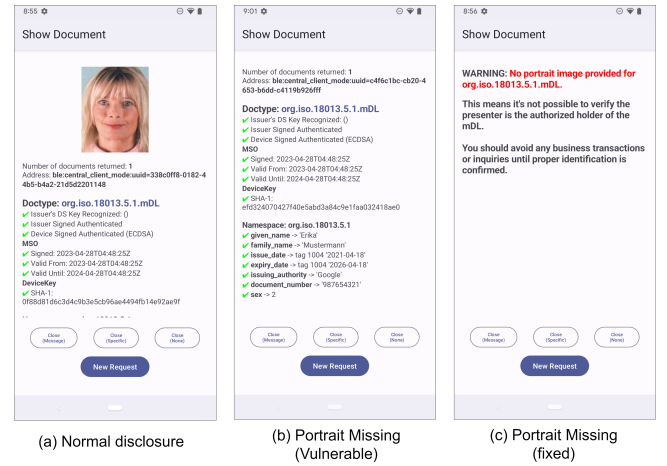
Verify that the "DocType" in the MSO matches the relevant "DocType" in the "Documents" structure.

– BS ISO/IEC 18013-5:2021, section 9.3.1, page 59

Failure to do so could enable an attacker to verify themselves using a forged document type. For instance, a verifier looking for a specific document like a School ID might mistakenly accept a fraudulent Vaccination Card as a valid School ID. This can happen if an attacker alters the `docType` of a valid Vaccination Card to match a School ID and presents it to a verifier who does not check the consistency between the two `docType` values. Though data items in different documents might have separate namespaces, making direct borrowing difficult, an attacker could present a "blank" School ID. This could lead the verifier to mistakenly assume a valid school ID exists. We identified this issue in two mDL reader apps.

8.3 Case Study: Google Reference App

Identity Credential [20] is an open-source project that comprises of libraries and reference applications (i.e., reader and holder apps) associated with the ISO/IEC 18013-5:2021 and other relevant standards. The reference applications within this project are designed to showcase the functionality of the library APIs and exemplify best practices for their implementation.

**Figure 4: Portrait visualization in the Google Reference App**

In our analysis, we found that the reference applications do not meet the mDL standard and the mDL guidelines in checking the MSO's expiration status and do not follow the best practices when handling missing portraits. The reader app did not show any warning when the portrait was withheld by the mDL holder, hindering identity confirmation. To fix this, we submitted a pull request addressing the "Optional Portrait" issue, which was accepted.

Another problem we identified was the reference applications' failure to verify the expiration status of an MSO. The reader app merely confirmed the correctness of the `ValidityInfo`, such as ensuring that the `validUntil` is later than the `validFrom`. To address this, we submitted an issue to the GitHub repository. The maintainer acknowledged the issue and implemented a fix in the new version of Identity Credential, which is called Multipaz [30] and belongs to the OpenWallet Foundation.

8.4 Responsible Disclosure

We responsibly disclosed issues in the Google Reference App, GET Mobile Verify, and Tap2iD Mobile applications, as explained in Section 8.3. For the other three mDL reader apps listed in Table 3, we also reported our findings via email and offered assistance if needed, and we are currently waiting for an answer. Additionally, we met with the ISO committee presenting our findings and discussing

mitigation strategies, such as improving implementation guidelines to include a checklist of implementation requirements (Section 3).

9 Limitations and Future Work

Focus on Reader Apps. This study focuses on mDL reader apps and does not examine the implementation of mDL holder apps. This limitation is significant when evaluating the holder’s privacy. Security goals such as protection against eavesdropping and unauthorized access were modeled in our formal verification framework but not evaluated through our automated dynamic testing of real-world mDL reader apps. Future work should investigate mDL holder apps and evaluate their privacy protection mechanisms.

Focus on Device Retrieval. Our study primarily focuses on device retrieval. Hence, we do not consider the server retrieval mechanism, which allows a verifier to obtain the holder’s information directly from the IA, using an Internet connection. We leave the analysis of this functionality as future work.

Focus on Public Android Apps. Our study’s scope is limited to public Android mDL reader apps, excluding iOS apps (e.g., Apple Wallet) and private/enterprise apps unavailable publicly. While our methodology concentrates on public Android apps, the underlying principles and analytical approach could potentially be adapted and extended in future work to investigate the security and privacy aspects of iOS and private reader apps.

Human Factor. While our analysis detects UI differences in reader apps for non-compliant mDLs, these differences do not guarantee that the human verifier would react appropriately to such UI changes. As future work, we could conduct usability studies to determine how human verifiers react to specific UI cues in mDL reader apps and how to design UIs that are both secure and usable.

Evolving standard. The mDL standardization landscape is rapidly evolving. As ISO/IEC 18013-7:2025 was not yet available at the time of writing, this study analyzes the ecosystem based on the preceding standards. We acknowledge this limitation and highlight the need for future research to evaluate the implications of the newly released specification.

In this vein, ISO/IEC 18013-5:2021 standard does not mandate authentication on the mDL holder’s device itself to verify the identity of the user presenting the mDL. For this reason, we considered this aspect out of scope for our analysis. Future work should focus on analyzing the security of the standard in the presence of such mechanisms, which attempt to link the identity of the holder and the identity of the presenter.

10 Related Work

Analyzing the Security of Electronic Identity. Electronic Identity (eID) cards and systems have been widely studied in the literature. Laud and Roos [25] analyzed the Estonian Mobile-ID protocol for its resilience against network attacks, suggesting improvements. Our paper extends these concepts to mDL, focusing on its unique security challenges and goals. Christofi and Gouget [15] discussed an mERA-based protocol with a trusted third party, highlighting the merits of formal verification in terms of full authentication. Blynkov and Yaremenko [12] analyzed the security of various mDL storage techniques and identified potential issues using utilizing Trusted Execution Environments for mDLs. Lelyak and Astrakhantsev [28]

discuss an alternative to NFC usage for mDLs by introducing a secure form of BLE and Wi-Fi Aware data transfer that utilizes ultrawide band to ensure that the legitimate holder and reader devices are communicating with each other.

ProVerif Modeling and Querying Security Properties. Formal verification methods are extensively used to evaluate systems and protocols. Dreier et al. [18] presented a framework for e-cash system analysis using applied π -calculus and ProVerif to ensure authenticity. Lei et al. [26] analyzed the vulnerability of SMS OTPs to local attacks via reverse engineering, formal verification, user studies, and automated analysis. Our paper applies similar methods to mDL, focusing on its security and real-world scenarios. Wu et al. [35] utilized ProVerif for Bluetooth protocol vulnerability analysis, focusing on modular design for flexibility and reuse. Our study shares this modular approach but extends it with specialized ProVerif modules. Imran et al. [22] created ProVerif models to show that existing Android techniques to perform remote authorization were not secure and then created an alternative protocol whose security was formally verified.

Verifying Credential Systems with Selective Disclosure. Selective disclosure of attributes is a fundamental element of privacy-preserving systems, such as the mDL. Wang et al. [33, 34] used applied π -calculus in their framework to verify security in selective disclosure attribute-based credential systems, focusing on the U-Prove protocol [29]. Our mDL paper, while sharing some similarities, differs by concentrating on the specific mDL protocol rather than U-Prove.

Differential Analysis for Security Implementation. Differential analysis has proven effective for security verification by comparing system behaviors across inputs. Previous work includes Brubaker et al. [13]’s Frankencerts for uncovering SSL/TLS validation flaws, Continella et al. [16]’s application for privacy leak detection, and Lei et al. [27]’s verification of Android’s Scoped Storage. Our work extends prior approaches to mDL readers by combining runtime instrumentation with UI state comparison, systematically addressing false negatives for accurate flaw detection.

11 Conclusion

This paper presents the first comprehensive security analysis of the mDL standard. In particular, we implemented a formal verification framework to verify the correctness of the mDL standard and to study the consequences that specific implementation mistakes have on the security of the protocol in different real-world scenarios. Additionally, we developed mDLVerif, a dynamic analysis tool that automates the verification of mDL reader apps. Finally, we used mDLVerif to check for the presence of these implementation mistakes, and we identified several issues in real-world apps.

Acknowledgments

This work was partially supported by Google through its ASPIRE Research Award. We thank David Zeuthen, David Kleidermacher, René Mayrhofer of Google and Kevin Deus for their feedback on our paper.

References

- [1] 2020. *The Mobile Driver's License (mDL) and Ecosystem*. White Paper. The Security Technology Alliance, Princeton Junction, US. <https://www.securetechalliance.org/wp-content/uploads/Mobile-Drivers-License-WP-FINAL-Update-March-2020-4.pdf>
- [2] 2021. *Mobile Driver's License (mDL) Implementation Guidelines*. Technical Report. American Association of Motor Vehicle Administrators, Arlington, US. https://www.aamva.org/getmedia/c4fe2a21-91ff-449d-9df3-5a7e33cf3a8e/mDL-Implementation-Guidelines-1-0_2021.pdf
- [3] 2025. Attack demonstrations Based on the Vulnerabilities Identified by mDLVerif. <https://anonymous.4open.science/r/mdl-attack-videos-EB1C/>.
- [4] 2025. Frida. <https://frida.re>.
- [5] 2025. Gemini Deep Research. <https://deepresearch.google.com/>.
- [6] 2025. Source Code of the mDL Model Generation Framework and the Dynamic Testing Tool mDLVerif. <https://github.com/purseclab/mdl-proverif>.
- [7] 2025. Table 4 of ISO 18013-5 [Screenshot]. <https://anonymous.4open.science/r/MDL-ISO-standard/Security%20Mechanisms.png>.
- [8] 2025. UI Automator. <https://developer.android.com/training/testing/ui-automator>.
- [9] Privacy Affairs. 2022. Personal ID Selling for as Little as \$17 on Dark Web, Privacy Affairs Study Reveals. <https://www.prnewswire.com/news-releases/personal-id-selling-for-as-little-as-17-on-dark-web-privacy-affairs-study-reveals-301570978.html>.
- [10] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. AndroZoo: Collecting Millions of Android Apps for the Research Community. In *Proceedings of the 13th International Conference on Mining Software Repositories (Austin, Texas) (MSR '16)*. ACM, New York, NY, USA, 468–471. doi:10.1145/2901739.2903508
- [11] Bruno Blanchet et al. 2016. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends® in Privacy and Security* 1, 1-2 (2016), 1–135.
- [12] V Blynkov and V Yaremenko. 2020. Mobile driving license system deployment model with security enhancement. *Theoretical and Applied Cybersecurity* 2, 1 (2020).
- [13] Chad Brubaker, Suman Jana, Baishakhi Ray, Sarfraz Khurshid, and Vitaly Shmatikov. 2014. Using Frankencerts for Automated Adversarial Testing of Certificate Validation in SSL/TLS Implementations. In *Proceedings of the IEEE Symposium on Security and Privacy (SP)*. IEEE, 114–129.
- [14] Jon Callas. 2020. Privacy-preserving credentials for smartphones are coming. <https://www.brookings.edu/techshot/privacy-preserving-credentials-for-smartphones-are-coming/>.
- [15] Maria Christofi and Aline Gouget. 2012. Formal Verification of the mERA-Based eServices with Trusted Third Party Protocol. In *SEC*. Springer, 299–314.
- [16] Andrea Continella, Yanick Fratantonio, Martina Lindorfer, Federico Maggi, Stefano Zanero, Christopher Kruegel, and Giovanni Vigna. 2017. Obfuscation-Resilient Privacy Leak Detection for Mobile Apps through Differential Analysis. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
- [17] Credence ID. 2024. Tap2iD Mobile™. <https://play.google.com/store/apps/details?id=com.credenceid.phoneverifier>.
- [18] Jannik Dreier, Ali Kassem, and Pascal Lafourcade. 2015. Formal analysis of e-cash protocols. In *2015 12th International Joint Conference on e-Business and Telecommunications (ICETE)*, Vol. 4. IEEE, 65–75.
- [19] GET Group NA. 2025. GET Mobile Verify. <https://play.google.com/store/apps/details?id=com.getgroupna.mdl.pol>.
- [20] Google. 2025. Identity Credential. <https://github.com/google/identity-credential>. GitHub repository.
- [21] Jennifer Hagman, Ann Hendrickson, and Amanda Whitty. 2003. What's in a barcode? informed consent and machine scannable driver licenses. In *CHI'03 Extended Abstracts on Human Factors in Computing Systems*. 912–913.
- [22] Abdullah Imran, Habiba Farrukh, Muhammad Ibrahim, Z Berkay Celik, and Antonio Bianchi. 2022. {SARA}: Secure android remote authorization. In *31st USENIX Security Symposium (USENIX Security 22)*. 1561–1578.
- [23] ISO/IEC 18013-5:2021 2021. *Personal identification - ISO-compliant driving licence - Part 5: Mobile driving licence (mDL) application*. Standard. International Organization for Standardization, Geneva, CH. <https://www.iso.org/standard/69084.html>
- [24] Neeraj Kumar, Alexander C Berg, Peter N Belhumeur, and Shree K Nayar. 2009. Attribute and simile classifiers for face verification. In *2009 IEEE 12th international conference on computer vision*. IEEE, 365–372.
- [25] Peeter Laud and Meelis Roos. 2009. Formal analysis of the estonian mobile-ID protocol. In *Identity and Privacy in the Internet Age: 14th Nordic Conference on Secure IT Systems, NordSec 2009, Oslo, Norway, 14-16 October 2009. Proceedings 14*. Springer, 271–286.
- [26] Zeyu Lei, Yuhong Nan, Yanick Fratantonio, and Antonio Bianchi. 2021. On the insecurity of SMS one-time password messages against local attackers in modern mobile devices. In *Network and Distributed Systems Security (NDSS) Symposium 2021*.
- [27] Zeyu Lei, Güliz Seray Tuncay, Beatrice Carissa Williem, Z Berkay Celik, and Antonio Bianchi. 2025. ScopeVerif: Analyzing the Security of Android's Scoped Storage via Differential Analysis. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
- [28] Andriy Valeriyovich Lelyak and Andrey Anatolyevich Astrakhantsev. 2023. Improve mobile driving license data transfer security via Ble/Wi-Fi aware with UWB ranging. *Telecommunications problems* 2 (33) (2023), 62–74.
- [29] Wojciech Mostowski and Pim Vullers. 2012. Efficient U-Prove implementation for anonymous credentials on smart cards. In *Security and Privacy in Communication Networks: 7th International ICST Conference, SecureComm 2011, London, UK, September 7-9, 2011, Revised Selected Papers 7*. Springer, 243–260.
- [30] OpenWallet Foundation. 2025. Multipaz SDK for Digital Credentials. <https://github.com/openwallet-foundation/multipaz>.
- [31] Help Net Security. 2022. The price of stolen info: Everything on sale on the dark web. <https://www.helpnetsecurity.com/2022/06/22/stolen-info-sale-dark-web/>.
- [32] staff report. 2019. Indiana State Police looks for police impersonator. <https://www.aura.com/learn/can-someone-steal-your-identity-with-your-id>.
- [33] Caimei Wang, Yan Xiong, Wenjuan Cheng, Wenchao Huang, Huihua Xia, and Jianmeng Huang. 2017. A General Formal Framework of Analyzing Selective Disclosure Attribute-Based Credential Systems. *Int. J. Netw. Secur.* 19, 5 (2017), 794–803.
- [34] Caimei Wang, Yan Xiong, Wenchao Huang, Huihua Xia, and Jianmeng Huang. 2016. Formal analysis of selective disclosure attribute-based credential system in applied pi calculus. In *2016 IEEE Trustcom/BigDataSE/ISPA*. IEEE, 42–49.
- [35] Jianliang Wu, Ruoyu Wu, Dongyan Xu, Dave Jing Tian, and Antonio Bianchi. 2022. Formal Model-Driven Discovery of Bluetooth Protocol Design Vulnerabilities. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2285–2303.

A mDL Usage Scenarios and Requirements

Table 4 shows the 11 real world usage scenarios of mDL standard as listed in the STA white paper [1] along with their business questions and the respective ProVerif queries.

Table 5 summarizes the seven critical implementation requirements for mDL reader apps, extracted from the mDL standard specification with their corresponding page references and exact quotes.

Table 4: Verified Real-world Scenarios

Ref	Scenario	Business Questions
1	Confirming Driving Privilege	• Has the driver earned and retained the privilege of being able to drive a certain type of vehicle?
2	Identifying the Driver	• Who is the driver of the vehicle?
3	Entering a Casino	• Is the customer old enough? • Is the customer's full name not listed on the no-gamble list?
4	Purchasing Age-Restricted Items	• Is the customer old enough?
5	Renting Cars	• Is the customer older enough to rent a car? • Has the customer earned and retained driving privilege?
6	Checking into a Hotel	• Is the customer old enough to rent a hotel room? • Who is the customer?
7	Accessing Secure Buildings	• Who is the visitor?
8	Airline Check-in Counter	• Does the customer's full name matches the name on the boarding pass
9	Receiving Social Services	• Is the applicant a resident of this state?
10	Opening Bank Accounts	• Who is the applicant?
11	Entering Secure Area	• Has the visitor previously been granted access? • Who is the visitor?

* All scenarios can be found in Section 11 of the mDL white paper [1].

B Formal Model Implementation Details

Figure 5 illustrates the ten steps across three phases of the protocol implementation in our ProVerif model.

C Details on Identifying mDL Reader Applications

This appendix provides additional details on the methodology used to identify mDL reader applications.

Static Analysis. We first filtered out apps published before 2019 based on their update timestamps, as the mDL standard's first draft started in early 2020. Then, we analyzed the requested permissions of each app. Specifically, we first checked for the existence of the "Full network access" permission on the app page. Additionally, since the specification mandates an mDL app must support either NFC or BLE for data retrieval and either NFC or QR code for device engagement, we checked whether the app declares "Control Near Field Communication," or declares both "Take pictures and videos," and "Pair with Bluetooth devices." Applying these filtering steps on an initial dataset of 2,316,229 apps from AndroZoo [10] gave us 190,275 apps.

Next, we carried out a static analysis on the remaining dataset, which involved searching for specific strings that are indicative of mDL-related processes, such as "iso.18013.5," "issuerAuth," and

"deviceAuth." These strings should be present in both mDL reader and holder apps as they are necessary for parsing and encoding mDL objects, respectively. Through this analysis, we narrowed the list down to 237 candidate apps.

Finally, we conducted an in-depth manual analysis, identifying 13 mDL apps, two of which are mDL reader apps. Additionally, we discovered an open-source library from Google [20], called "identity-credential," designed to support third-party developers in implementing mDL apps and to demonstrate best practices.

In summary, we identified three mDL reader apps (two real-world apps and Google library) from static and manual analysis.

Gemini Deep Research. In parallel, we utilized the Gemini Deep Research [5] tool to further identify mDL reader apps. We used this tool to search the web for mDL reader apps and perform reasoning to filter out false positives using app descriptions, reviews, and other available information. Through this approach, we identified five additional mDL reader apps.

D Additional Case Studies

GET Mobile Verify.

GET Mobile Verify [19] is an mDL reader application developed by GET Group. Recently, the app gained acceptance at TSA PreCheck. Our analysis revealed that the GET Mobile Verify app is vulnerable to docType Forgery due to its failure to perform the docType check. By manipulating the docType, a forged document, like a car registration, could be presented as a driver's license. Another problem we identified is that the app fails to verify whether a portrait has been provided. We observed that the GET Mobile Verify app does not alert the user about this issue, and it just shows a white placeholder in place of the holder's picture.

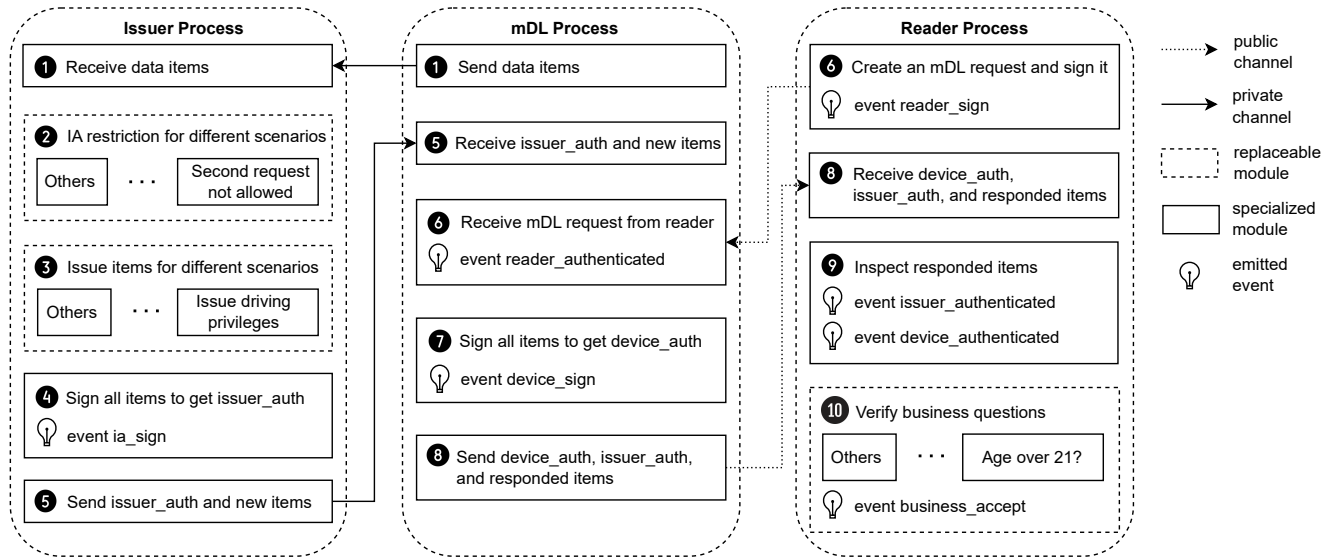
The GET Group, after being informed about issues in their application, engaged in a discussion with us and subsequently resolved the "Expired MSO" issue in their app's latest version. They also acknowledged both the "Optional Portrait" problem and "docType Forgery" issue and began working on a solution.

CredenceID. Tap2iD Mobile [17] is an mDL reader app developed by CredenceID, LLC. Through reverse engineering and static analysis, we initially confirmed that it implemented most of the mDL implementation requirements. However, mDLVerif confirmed dynamically that it fails to check "mdoc authentication", "trust anchor", "MSO signature", and "item digests". After investigating the issue, we found that the app indeed implemented these checks, but its UI did not use the verification results accordingly.

Even if the app internally checks these non-compliant mDLs, human verifiers can still not distinguish between compliant and non-compliant mDLs. While this issue is very hard to detect through static analysis, mDLVerif can catch this issue automatically since it uses the final rendered UI as the ground truth. We have reported this issue to CredenceID and received their acknowledgment.

Table 5: Critical implementation requirements (security checks) for mDL reader apps, extracted from ISO/IEC 18013-5:2021

ID	Description	Pages	Quotes from ISO/IEC 18013-5:2021 [23]
C ₁	No mdoc Authentication	52–54	“The security objective of mdoc authentication is to prevent cloning of the mdoc and to mitigate man in the middle attacks.” (Section 9.1.3.1–9.1.3.2)
C ₂	No verification of the certificate chain	59–60	“The mdoc reader shall perform the following or functionally equivalent steps to verify that the received data is authentic and valid [...] 1. Validate the certificate included in the MSO header according to 9.3.3.” (Section 9.3.1, Step 1)
C ₃	Failure to authenticate the signature of MSO	59	“2. Verify the digital signature of the IssuerAuth structure (see 9.1.2.4) using the working_public_key, working_public_key_parameters, and working_public_key_algorithm from the certificate validation procedure of step 1.” (Section 9.3.1, Step 2)
C ₄	No checks for item digests	59	“3. Calculate the digest value for every IssuerSignedItem returned in the Documents structure according to 9.1.2.5 and verify that these calculated digests equal the corresponding digest values in the MSO.” (Section 9.3.1, Step 3)
C ₅	Failure to match docType	59	“4. Verify that the DocType in the MSO matches the relevant DocType in the Documents structure.” (Section 9.3.1, Step 4)
C ₆	Failure to check document expiration	59	“5. Validate the elements in the ValidityInfo structure, i.e. verify that: [...] the 'validUntil' element shall be equal or later than the current timestamp.” (Section 9.3.1, Step 5)
C ₇	Check if portrait matches the presenter	20	“From the set of data elements with mandatory presence [...], the portrait of the mDL holder is the only data item for verifying that the person presenting the mDL is the mDL holder.” (Section 7.2.1, Note 2)

**Figure 5: Formal model implementation details**